

UNIT – V

Unsupervised Learning : Clustering-K-means, K-Modes, K-Prototypes, Gaussian Mixture Models, Expectation-Maximization.

Reinforcement Learning: Exploration and exploitation trade-offs, non-associative learning, Markov decision processes, Q-learning.

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

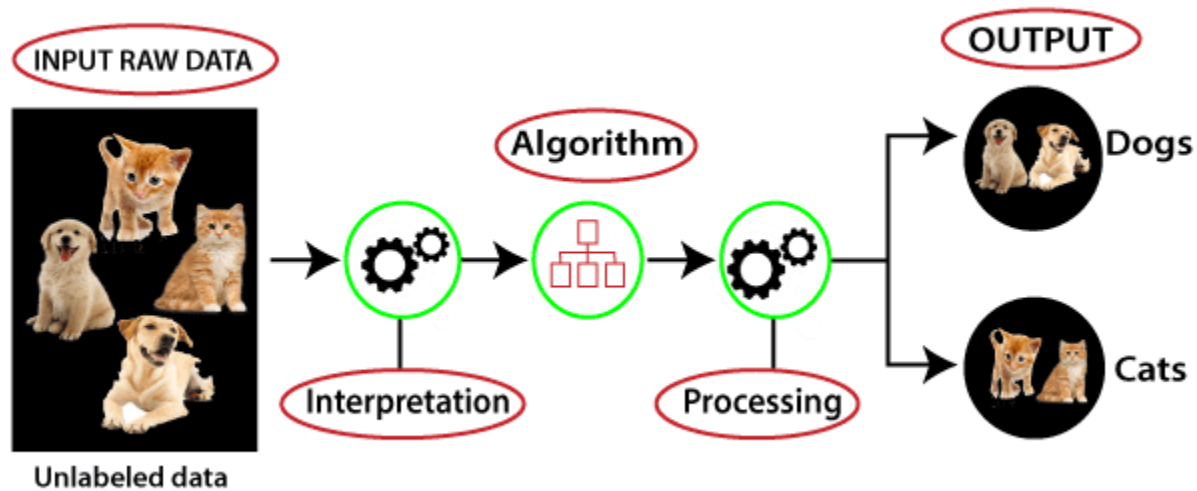
Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

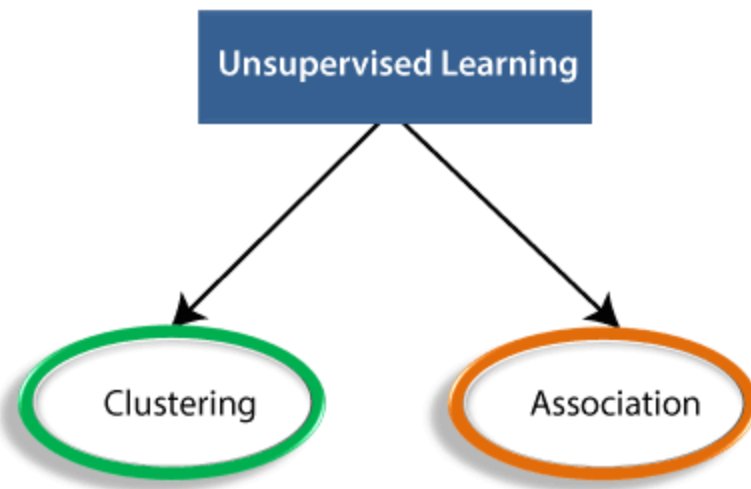


Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain in a group and have less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

K-means clustering

S.No	x	y
1	170	60
2	160	55
3	180	75
4	150	50
5	175	65
6	190	85

with $k=2$
Initial Centroids

c	x	y
c1	170	60
c2	160	55

$$\begin{aligned}\text{Euclidean distance} &= \sqrt{\sum_{i=1}^n (y_i - x_i)^2} \\ &= \sqrt{(x - x_i)^2 + (y - y_i)^2}\end{aligned}$$

For Row ③ ED from c₁ to ③

$$\begin{aligned}\text{ED} &= \sqrt{(180 - 170)^2 + (75 - 60)^2} \\ &= \sqrt{100 + 225} \\ &= 18.02\end{aligned}$$

ED from c₂ to ③

$$\begin{aligned}\text{ED} &= \sqrt{(180 - 160)^2 + (75 - 55)^2} \\ &= 28.28\end{aligned}$$

New Centroids				New Centroids		
C	x	y		C	x	y
c ₁	$\frac{170+180}{2}$	$\frac{75+60}{2}$	⇒	c ₁	175	67.5
c ₂	160	55		c ₂	160	55

c ₁ = { 1, 3 }	}
c ₂ = { 2 }	}

c ₁ to ④	
ED = $\sqrt{25^2 + 17.5^2}$	
= 30.51	
c ₂ to ④	
ED = $\sqrt{10^2 + 5^2}$	
= 11.18 (Minimum)	

So, Row-4 is near to cluster ④

c ₁ = { 1, 3 }	
c ₂ = { 2, 4 }	

C	x	y
c ₁	175	67.5
c ₂	$\frac{160+150}{2}$	$\frac{55+50}{2}$
	155	52.5

New Centroids		
C	x	y
c ₁	175	67.5
c ₂	155	52.5

c_1 to ⑤
 $ED = \sqrt{0 + 2.5^2}$
 $= 2.5$ (minimum)

c_2 to ⑤
 $ED = \sqrt{20^2 + 12.5^2}$
 $= 23.58$

$c_1 = \{1, 3, 5\}$
 $c_2 = \{2, 4\}$

new Centroids

c	x	y
c_1	$\frac{175+175}{2}$	$\frac{65+65}{2}$
c_2	155	52.5

⇒

c	x	y
c_1	175	66.25
c_2	155	52.5

c_1 to ⑥
 $ED = \sqrt{15^2 + 18.75^2}$
 $= 24.01$ (minimum)

c_2 to ⑥
 $ED = \sqrt{35^2 + 32.5^2}$
 $= 47.76$

$c_1 = \{1, 3, 5, 6\}$
 $c_2 = \{2, 4\}$

new Centroids

c	x	y
c_1	$\frac{170+175}{2}$	$\frac{65+66.25}{2}$
c_2	155	52.5

⇒

c	x	y
c_1	182.5	75.6
c_2	155	52.5

k	x	y
c_1	170	60
	180	75
	175	65
	190	85
c_2	160	55
	150	50

K-Mode Clustering

K-mode clustering is an unsupervised machine-learning technique used to group a set of data objects into a specified number of clusters, based on their categorical attributes. The algorithm is called “K-Mode” because it uses modes (i.e. the most frequent values) instead of means or medians to represent the clusters.

How Does the KModes Algorithm Work?

1. Pick K observations at random and use them as leaders/clusters
2. Calculate the dissimilarities and assign each observation to its closest cluster
3. Define new modes for the clusters
4. Repeat 2–3 steps until there are no re-assignment required

	Attr1	Attr2
1	1	A
2	2	B
3	3	B
4	1	A
5	1	C
6	2	C
7	3	C
8	1	C

	I	II	III	Which ?
Initial	(1,A)	(3,B)	(3,C)	
1	0	2	2	I
2	2	1	2	II
3	2	0	1	II
4	0	2	2	I
5	1	2	1	III
6	2	2	1	III
7	2	1	0	III
8	1	2	1	III

{1,4} {2,3} {5,6,7,8}

	Attr1	Attr2
1	1	A
4	1	A
2	2	B
3	3	B
5	1	C
6	2	C
7	3	C
8	1	C

New Mode		
I	II	III
(1,A)	(3,B)	(1,C)

Example: Imagine we have a dataset that has the information about hair color, eye color, and skin color of persons. We aim to group them based on the available information(maybe we want to suggest some styling ideas)

Hair color, eye color, and skin color are all categorical variables. Below is how our dataset looks like.

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Alright, we have the sample data now. Let us proceed by defining the number of clusters(K)=3

Step 1: Pick K observations at random and use them as leaders/clusters

I am choosing P1, P7, P8 as leaders/clusters

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Step 2: Calculate the dissimilarities(no. of mismatches) and assign each observation to its closest cluster

Iteratively compare the cluster data points to each of the observations. Similar data points give 0, dissimilar data points give 1.

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing leader/Cluster P1 to the observation P1 gives 0 dissimilarities.

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing leader/cluster P1 to the observation P2 gives $3(1+1+1)$ dissimilarities.

Likewise, calculate all the dissimilarities and put them in a matrix as shown below and assign the observations to their closest cluster(cluster that has the least dissimilarity)

	Cluster 1 (P1)	Cluster 2 (P7)	Cluster 3 (P8)	Cluster
P1	0 ✓	2	2	Cluster 1
P2	3 ✓	3	3	Cluster 1
P3	3	1 ✓	3	Cluster 2
P4	3	3	1 ✓	Cluster 3
P5	1 ✓	2	2	Cluster 1
P6	3	3	2 ✓	Cluster 3
P7	2	0 ✓	2	Cluster 2
P8	2	2	0 ✓	Cluster 3

Dissimilarity matrix (Image by Author)

After step 2, the observations P1, P2, P5 are assigned to cluster 1; P3, P7 are assigned to Cluster 2; and P4, P6, P8 are assigned to cluster 3.

Note: If all the clusters have the same dissimilarity with an observation, assign to any cluster randomly. In our case, the observation P2 has 3 dissimilarities with all the leaders. I randomly assigned it to Cluster 1.

Step 3: Define new modes for the clusters

Mode is simply the **most observed value**.

Mark the observations according to the cluster they belong to. Observations of Cluster 1 are marked in Yellow, Cluster 2 are marked in Brick red, and Cluster 3 are marked in Purple.

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Looking for Modes (Image by author)

Considering one cluster at a time, for each feature, look for the Mode and update the new leaders.

Explanation: Cluster 1 observations(P1, P2, P5) has brunette as the most observed hair color, amber as the most observed eye color, and fair as the most observed skin color.

Note: If you observe the same occurrence of values, take the mode randomly. In our case, the observations of Cluster 3(P3, P7) have one occurrence of brown, fair skin color. I randomly chose brown as the mode.

Below are our new leaders after the update.

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown

Obtained new leaders

Repeat steps 2–4

After obtaining the new leaders, again calculate the dissimilarities between the observations and the newly obtained leaders.

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing Cluster 1 to the observation P1 gives 1 dissimilarity.

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing Cluster 1 to the observation P2 gives 2 dissimilarities.

Likewise, calculate all the dissimilarities and put them in a matrix. Assign each observation to its closest cluster.

	Cluster 1	Cluster 2	Cluster 3	Cluster
P1	1 ✓	2	3	Cluster 1
P2	2 ✓	3	2	Cluster 1
P3	3	1 ✓	2	Cluster 2
P4	3	3	0 ✓	Cluster 3
P5	0 ✓	2	3	Cluster 1
P6	3	3	1 ✓	Cluster 3
P7	2	0 ✓	3	Cluster 2
P8	2	2	1 ✓	Cluster 3

The observations P1, P2, P5 are assigned to Cluster 1; P3, P7 are assigned to Cluster 2; and P4, P6, P8 are assigned to Cluster 3.

We stop here as we see there is no change in the assignment of observations.

The K-Prototypes Algorithm

The K-Prototypes algorithm is almost similar to the k-means or k-modes clustering algorithm. The steps in the k-prototype algorithm are as follows.

1. First, we take K data points from the input dataset and use them as prototypes. The clusters are built around each prototype.
2. Find the distance of each data point from the current prototypes. Here, the distance is calculated according to the method discussed in the previous section.
3. After finding the distance of each data point from the prototypes, we assign data points to clusters. Here, each data point is assigned to the cluster with the prototype nearest to the data point.
4. After assigning data points to the clusters, we calculate new prototypes for each cluster. To calculate the prototypes, we take the mean of numeric attributes and the mode of categorical attributes.
5. If the new prototypes are the same as the previous prototypes, we say that the algorithm has converged. Hence, the current clusters are finalized. Otherwise, go to 2.

What Is K-Prototypes Clustering?

K-Prototypes clustering is an unsupervised machine learning algorithm. It is an ensemble of [k-means clustering](#) and k-modes clustering algorithms.

Being an ensemble of k-means clustering and [k-modes clustering](#), the k-prototypes clustering algorithm is used to perform clustering on a dataset with mixed data types. In other words, We can perform clustering on a dataset having numerical and categorical data using the k-prototypes clustering.

Distance Measures in K-Prototypes Clustering

The K-Prototypes clustering algorithm uses different distance measures for numerical and categorical attributes for any given data.

Distance Measure for Numerical Attributes

For numerical attributes, the k-prototypes clustering algorithm uses squared euclidean distance as the distance measure. If you have two records as (1, 2, 3) and (4,1, 5), the squared euclidean distance is calculated as $(1-4)^2 + (2-1)^2 + (3-5)^2$ which is equal to 14.

Distance Measure for Categorical Attributes

For categorical attributes, the kprototypes clustering algorithm follows matching dissimilarity. If you have two records (A, B, C, D) and (A, D, C, C) with categorical attributes, the matching dissimilarity is the number of different values at each position in the records. In the given records, values are different at the two positions only. Hence, the matching dissimilarity between the records will be 2.

Distance Measure for Records With Mixed Data Types

For records with mixed attributes, the kprototypes clustering algorithm uses both euclidean and matching dissimilarity. In this case, the algorithm calculates squared euclidean distance for numerical attributes and matching dissimilarity for categorical attributes and uses their resultant to calculate the distance between records having mixed attributes.

For instance, if we have two records ['A', 'B', 'F', 155, 53] and ['A', 'A', 'M', 174, 70].

- To find the distance between these two records, we will first find the dissimilarity score between ['A', 'B', 'F'] and ['A', 'A', 'M']. The score is 2 as two attributes out of three have different values.
- Next, we will calculate the square euclidean distance between [155, 53] and [174, 70]. Here, $(155-174)^2 + (53-70)^2$ which is equal to 650.
- Now, we can directly calculate the total dissimilarity score as the sum of the dissimilarity score between categorical attributes and the square euclidean distance of numerical attributes. Here, the sum will be equal to $650+2=652$.

Now, observe that the matching dissimilarity score of categorical attributes is almost negligible compared to the square euclidean distance between numerical attributes. Hence, the categorical attributes will have little or no effect on clustering.

To solve this problem, we can scale the values in numeric attributes within a range of say 0 to 5. Alternatively, we can take a weighted sum of the matching dissimilarity scores and the square euclidean distance. In this article, we will discuss a numerical example of the k-prototypes clustering algorithm by scaling the values in numeric attributes within a range of 0 to 5.

Selection of New Prototypes in the Clusters

Once a cluster is formed, we need to calculate a new prototype for the cluster using the data points in the current cluster.

To calculate the new prototype for any given cluster, we will take the mode of categorical attributes of the data points in the cluster. For numerical attributes, we will use the mean of the values to calculate a new prototype for the cluster. For example, suppose that we have the following data points in a cluster.

C	A	F	5.000000	4.705882
B	A	M	4.175824	4.470588
B	A	F	4.807692	3.235294
A	A	F	4.972527	4.117647
B	C	F	4.642857	3.470588

A sample cluster

The first three attributes of the cluster are categorical. Hence, we will calculate the mode of those attributes to obtain the prototype. For numeric attributes, we will use mean of the values. The prototype for the above cluster will be [B, A, F, 4.71978, 3.999999].

Gaussian Mixture Models (GMM)

A Gaussian mixture model can be used for clustering, which is the task of grouping a set of data points into clusters.

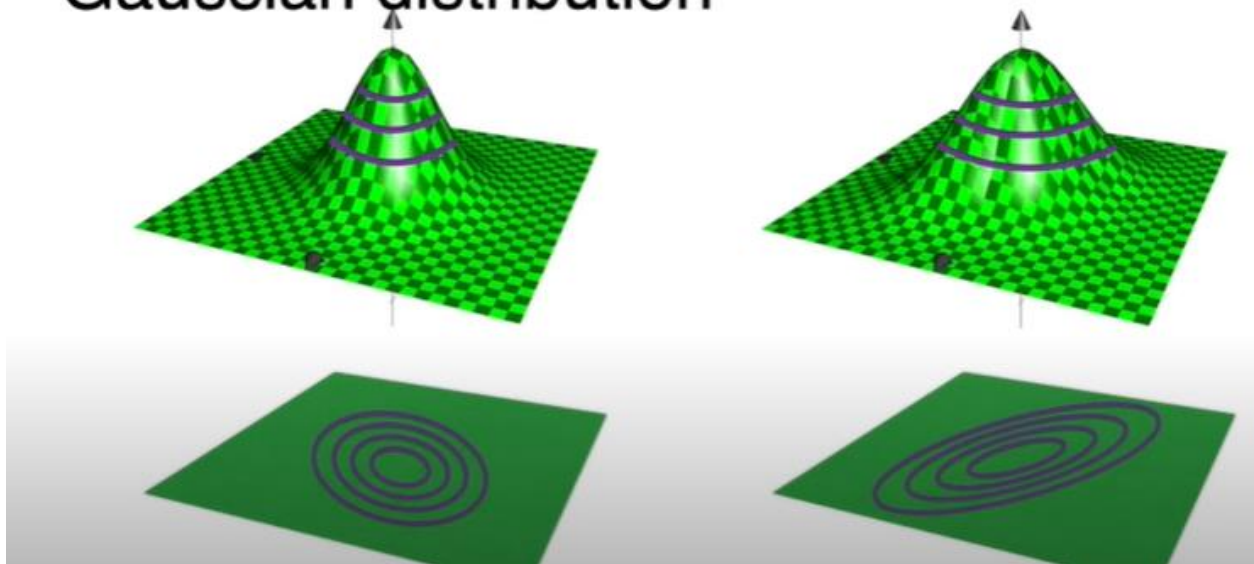
GMMs can be used to find clusters in data sets where the clusters may not be clearly defined. Additionally, GMMs can be used to estimate the probability that a new data point belongs to each cluster.

Gaussian mixture models are also relatively robust to outliers, meaning that they can still yield accurate results even if there are some data points that do not fit neatly into any of the clusters. This makes GMMs a flexible and powerful tool for clustering data.

The Gaussian mixture model can be understood as a probabilistic model where Gaussian distributions are assumed for each group and they have means and covariances which define their parameters.

GMM consists of two parts – mean vectors (μ) & covariance matrices (Σ). Recall that a Gaussian distribution is defined as a continuous probability distribution that takes on a bell-shaped curve. Another name for Gaussian distribution is the normal distribution.

Gaussian distribution



Gaussian distribution & their shadows.

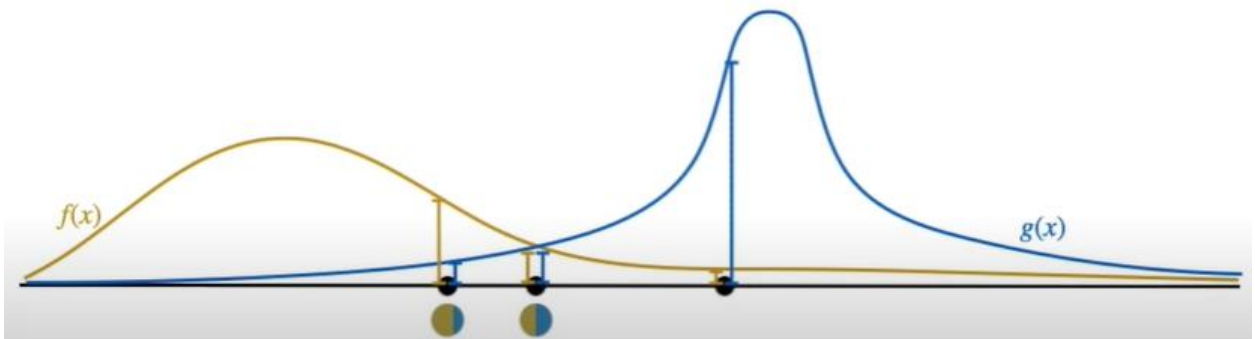
Step1:color points according to a gaussian.



Every point on the plane colored as these two blue&yellow.



Move towards to right points become blue.

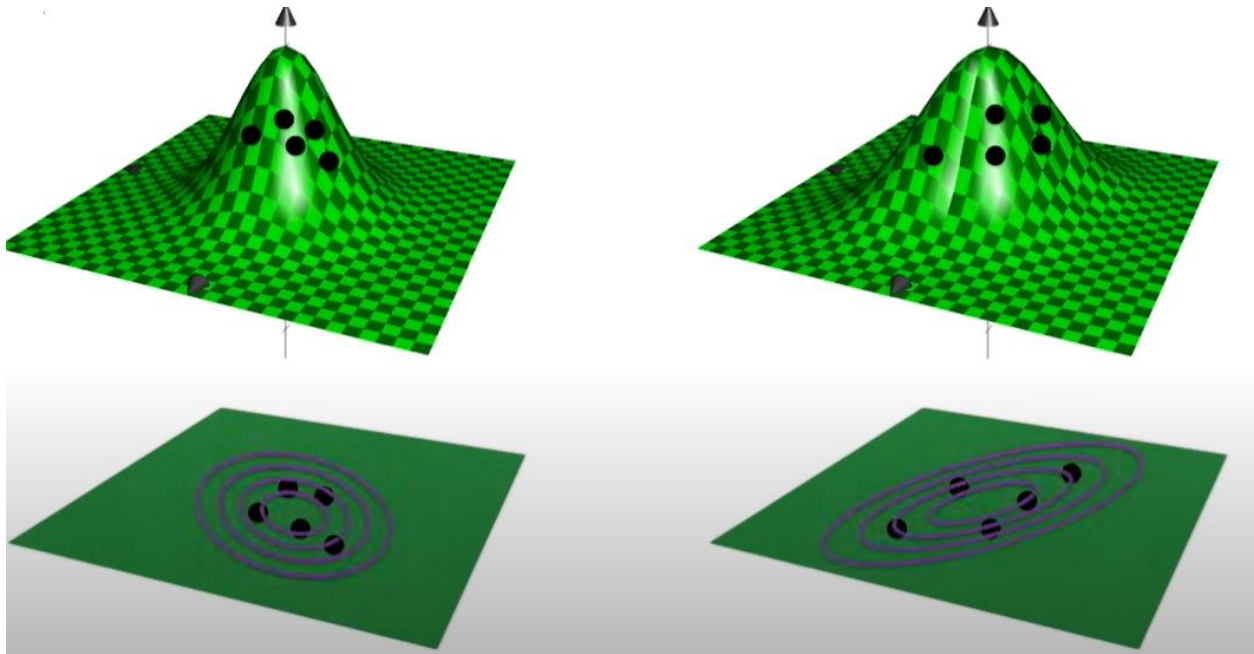


First point-→large distance from yellow curve so it is filled with yellow.

Second point 50-50 so half yellow, half blue.

Third point large distance from blue curve so it is filled with blue

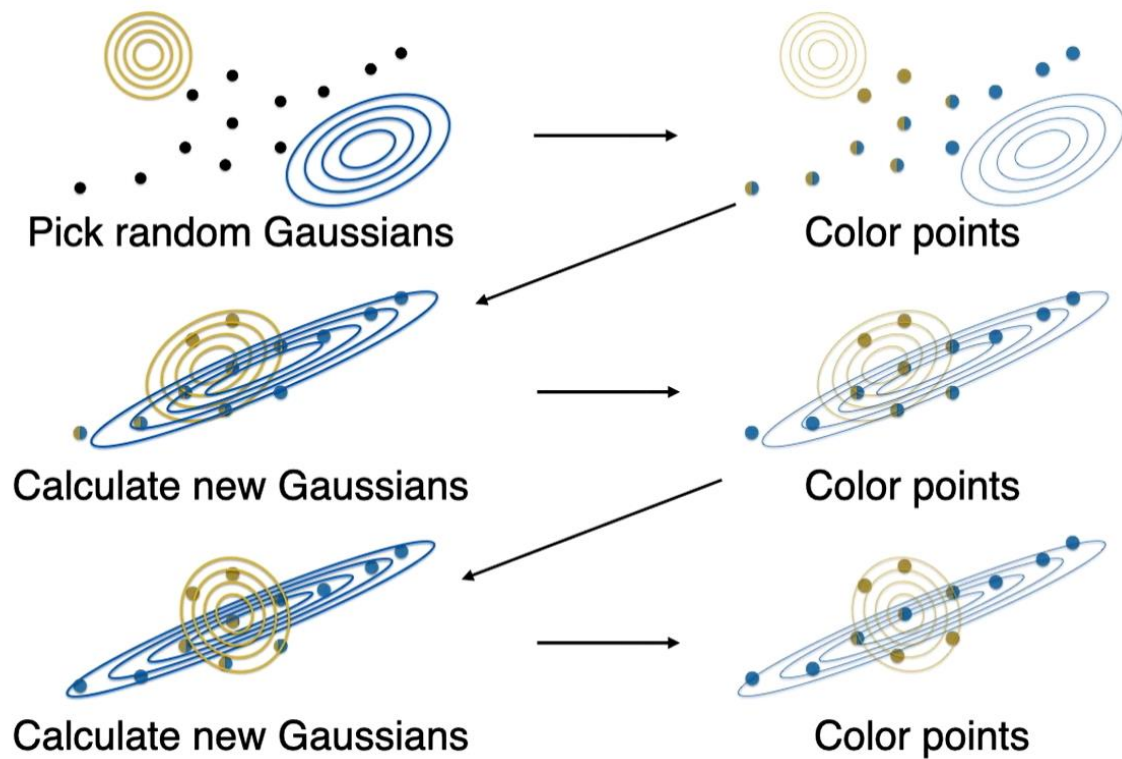
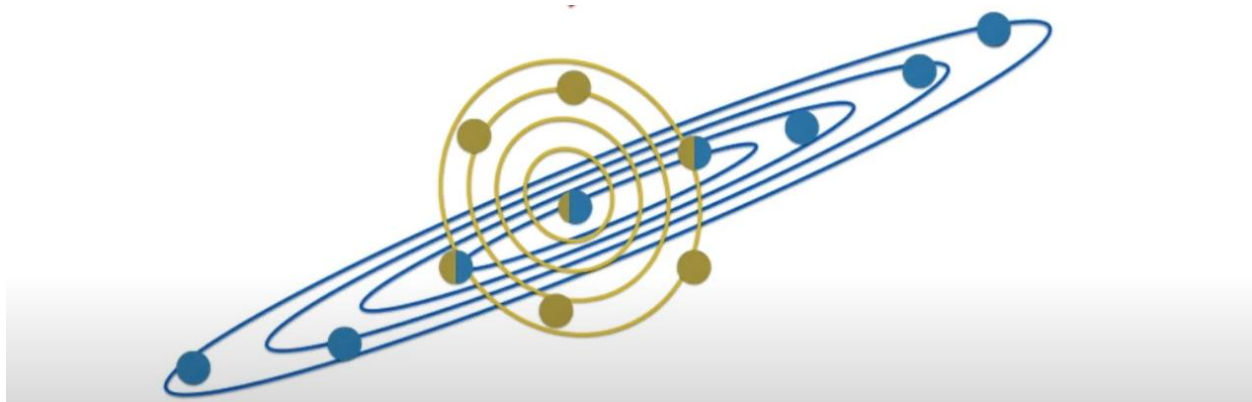
Step2:fitting a Gaussian



Points are fitted to 2 gaussians.

GMM algorithm:

1. color points according to Gaussians.
2. recalculate Gaussians according to colors.



So pick random Gaussians & color points then calculate new Gaussians & color points. This process continues until all points inserted into clusters.

Expectation - maximisation algorithm

- used to find Latent Variable - (not directly observed)
- basic for many unsupervised clustering algorithms.

→ How to get Latent Variable Values?

Any from previously observed Variables (depend on some other Variables to observe latent Variables).

Steps of EM algorithm

1. Initially a Set of initial Values are Considered a Set of incomplete data is given to System.

2. Expectation step:- we use observed data to estimate or guess the Values (based on previous observed data) of missing/incomplete data

3. Maximisation step:

we use Complete data generated in preceding Expectation step to update the Values.

4. we check if Values are Converging (not if Converging - stop

otherwise repeat Step-2, 3 till the Convergence occurs.

usage:

- 1) used to fill missing data
- 2) used for unsupervised clustering
- 3) used to discover values of latent variables.

Advantages:-

- With each iteration, likelihood increases, many convergence keep on reducing.
- Expectation step & Maximization step are easy to implement.

Disadvantages:-

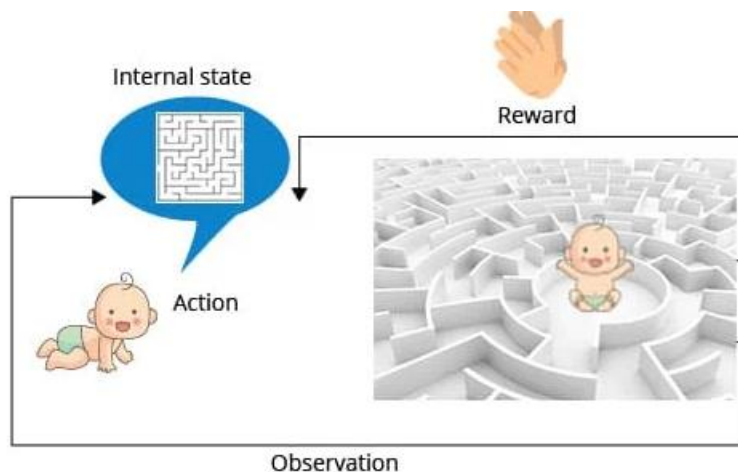
- 1) slow convergence, so many iterations needed.
- 2) makes convergence to local optimal only not global optimal.

Reinforcement Learning

Reinforcement Learning

child trying to take his/her first steps. What will be the instructions he/she follows to start walking?

1. **Observing** others walking and trying to replicate the same
2. **Standing still**
3. **Remaining still**
4. Trying to **balance** the body weight, along with deciding on which foot to advance first to start walking. It sounds like a difficult and challenging task for a child to get up and walk, right? But for us, it is easy since we have become used to it over time.



Now, putting it together, **a child is an agent** who is trying to manipulate the **environment** (surface or floor) by trying **to walk** and going from **one state to another** (taking a step). A child **gets a reward** when he/she takes a **few steps** (appreciation) but will **not receive any reward or appreciation** if he/she is unable to walk. This is description of a **reinforcement learning problem**.

Criteria	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Definition	The machine learns by using labeled data	The machine is trained on unlabeled data without any guidance	An agent interacts with its environment by performing actions & learning from errors or rewards
Type of problems	Regression & classification	Association & clustering	Reward-based
Type of data	Labeled data	Unlabeled data	No predefined data
Training	External supervision	No supervision	No supervision
Approach	Maps the labeled inputs to the known outputs	Understands patterns & discovers the output	Follows the trial-and-error method

Exploration and exploitation tradeoff

- *Exploration* is exploring the environment by trying random actions in order to find more information about the environment.
- *Exploitation* is exploiting known information to maximize the reward.



In this game, our mouse can have an **infinite amount of small cheese** (+1 each). But at the top of the maze, there is a gigantic sum of cheese (+1000).

However, if we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit **the nearest source of rewards**, even if this source is small (exploitation).

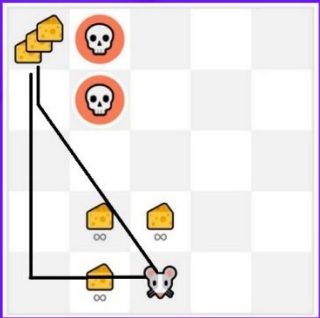
But if our agent does a little bit of exploration, it can **discover the big reward** (the pile of big cheese).

This is what we call the exploration/exploitation trade-off. We need to balance how much we **explore the environment** and how much we **exploit what we know about the environment**.

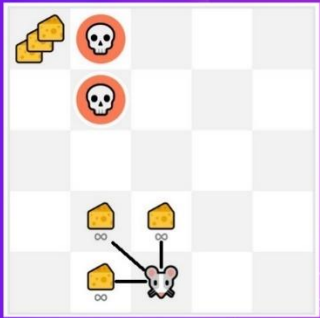
- *Exploitation*: You go to the same one that you know is good every day and **take the risk to miss another better restaurant**.
- *Exploration*: Try restaurants you never went to before, with the risk of having a bad experience **but the probable opportunity of a fantastic experience**.

Exploration/ Exploitation tradeoff

Exploration: trying **random actions** in order to find more **information about the environment**.



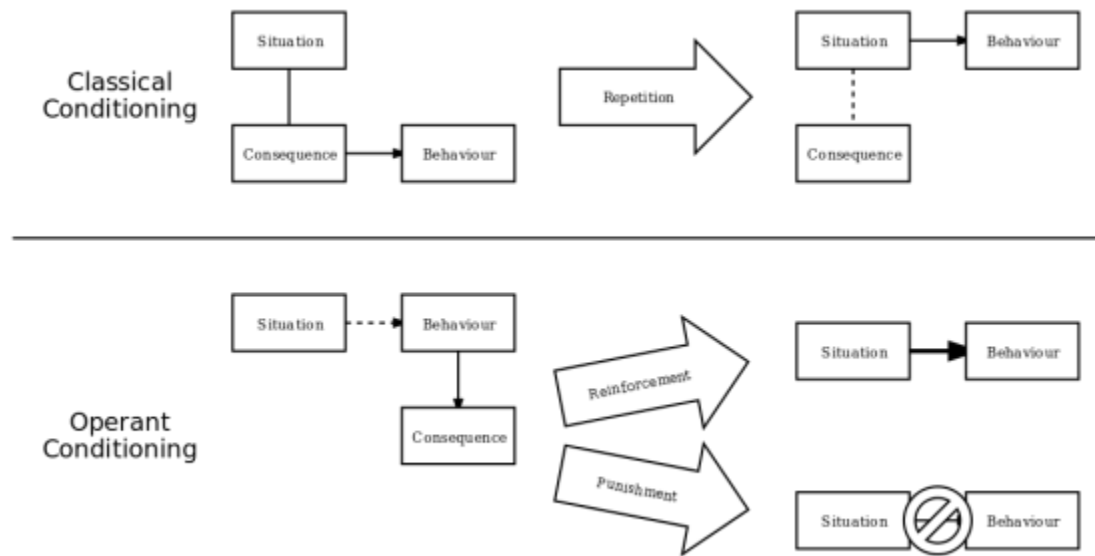
Exploitation: using known information to **maximize the reward**.



RL Course

Non Associative learning

Associative learning is a form of conditioning that involves a stimulus and a response. For example, when you eat a certain type of food, you may experience stomachaches. After you make the association between the food and your health, you will learn to avoid this food altogether. Similarly, as a child, you may have noticed that good grades get you praise and rewards, so you would have tried your best to achieve good grades.



Non-associative learning is a type of learning where there is no association between a stimulus and a behavior. It's a very simple form of learning. In non-associative learning, an organism's behavior toward a certain stimulus changes over time in the absence of any evident association with consequences or other stimuli that would induce such change. Moreover, habituation and sensitization are the two basic non-associative learning methods.

[Habituation](#) is a decrease in an innate response to a frequently repeated stimulus. For instance, if you are working with a radio playing in the background, the noise will distract you at first. But after a while, you will gradually tune out the noise and focus on your work. [Sensitization](#), on the other hand, is the increased reaction to a stimulus after repeated exposure to that stimulus. In this instance, you become more sensitive to the stimulus as time goes on. Here, frequent exposure to a stimulus increases the strength of the reaction to a stimulus.

Markov decision processes

Reinforcement Learning is defined by a specific type of problem, and all its solutions are classed as Reinforcement Learning algorithms. In the problem, an agent is supposed to decide the best action to select based on his current state.

When this step is repeated, the problem is known as a **Markov Decision Process**.

A **Markov Decision Process (MDP)** model contains:

- A set of possible world states S .
- A set of Models.
- A set of possible actions A .
- A real-valued reward function $R(s,a)$.
- A policy the solution of **Markov Decision Process**.

States: S

Model: $T(S, a, S') \sim P(S' \mid S, a)$

Actions: $A(S), A$

Reward: $R(S), R(S, a), R(S, a, S')$

Policy: $\Pi(S) \rightarrow a$
 Π^*

Markov Decision Process

What is a State?

A **State** is a set of tokens that represent every state that the agent can be in.

What is a Model?

A **Model** (sometimes called Transition Model) gives an action's effect in a state. In particular, $T(S, a, S')$ defines a transition T where being in state S and taking

an action 'a' takes us to state S' (S and S' may be the same). For stochastic actions (noisy, non-deterministic) we also define a probability $P(S'|S,a)$ which represents the probability of reaching a state S' if action 'a' is taken in state S . Note Markov property states that the effects of an action taken in a state depend only on that state and not on the prior history.

What are Actions?

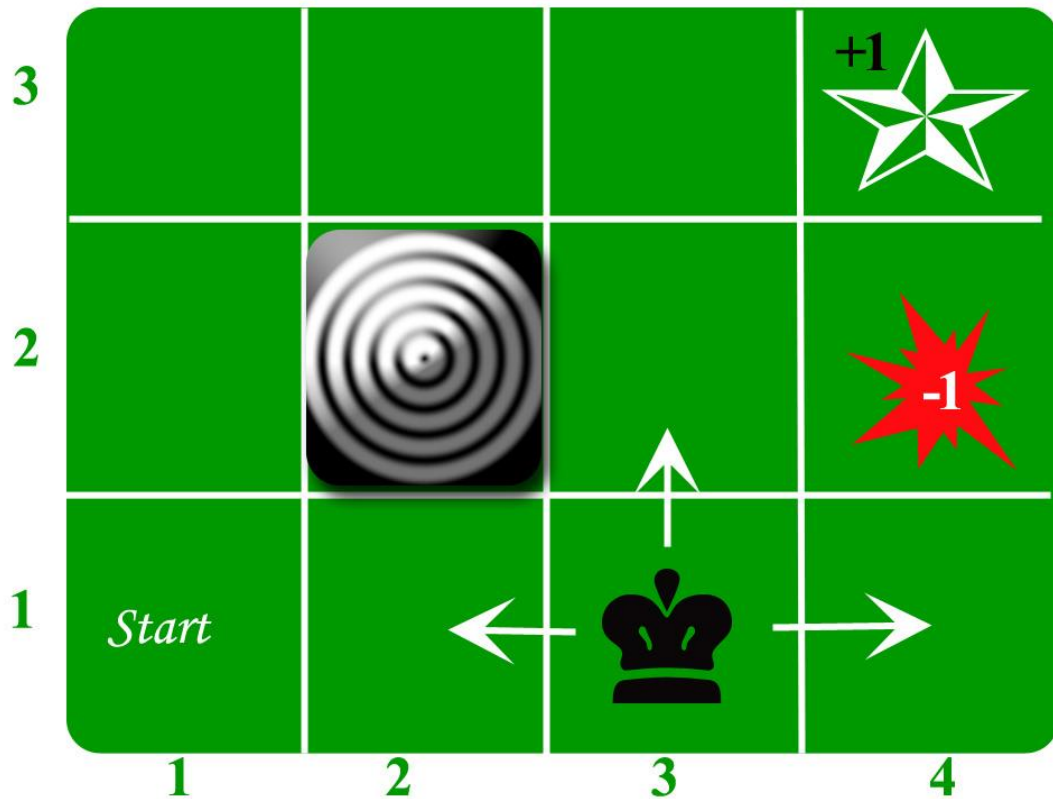
An **Action** A is a set of all possible actions. $A(s)$ defines the set of actions that can be taken being in state S .

What is a Reward?

A **Reward** is a real-valued reward function. $R(s)$ indicates the reward for simply being in the state S . $R(S,a)$ indicates the reward for being in a state S and taking an action 'a'. $R(S,a,S')$ indicates the reward for being in a state S , taking an action 'a' and ending up in a state S' .

What is a Policy?

A **Policy** is a solution to the Markov Decision Process. A policy is a mapping from S to a . It indicates the action 'a' to be taken while in state S . Let us take the example of a grid world:



An agent lives in the grid. The above example is a 3*4 grid. The grid has a START state(grid no 1,1). The purpose of the agent is to wander around the grid to finally reach the Blue Diamond (grid no 4,3). Under all circumstances, the agent should avoid the Fire grid (orange color, grid no 4,2). Also the grid no 2,2 is a blocked grid, it acts as a wall hence the agent cannot enter it.

The agent can take any one of these actions: **UP, DOWN, LEFT, RIGHT**
Walls block the agent path, i.e., if there is a wall in the direction the agent would have taken, the agent stays in the same place. So for example, if the agent says LEFT in the START grid he would stay put in the START grid.

First Aim: To find the shortest sequence getting from START to the Diamond.
Two such sequences can be found:

- **RIGHT RIGHT UP UPRIGHT**
- **UP UP RIGHT RIGHT RIGHT**

Let us take the second one (UP UP RIGHT RIGHT RIGHT) for the subsequent discussion.

The move is now noisy. 80% of the time the intended action works correctly. 20% of the time the action agent takes causes it to move at right angles. For

example, if the agent says UP the probability of going UP is 0.8 whereas the probability of going LEFT is 0.1, and the probability of going RIGHT is 0.1 (since LEFT and RIGHT are right angles to UP).

The agent receives rewards each time step:-

- Small reward each step (can be negative when can also be term as punishment, in the above example entering the Fire can have a reward of -1).
- Big rewards come at the end (good or bad).
- The goal is to Maximize the sum of rewards.

Q-learning

→ For each s, a initialize the table entry $\hat{Q}(s, a)$ to Zero

(s means state, a means action)

Every entry of table is fill with Zero

→ observe the current state s
from available set of states, select one state, assign this state as current state.

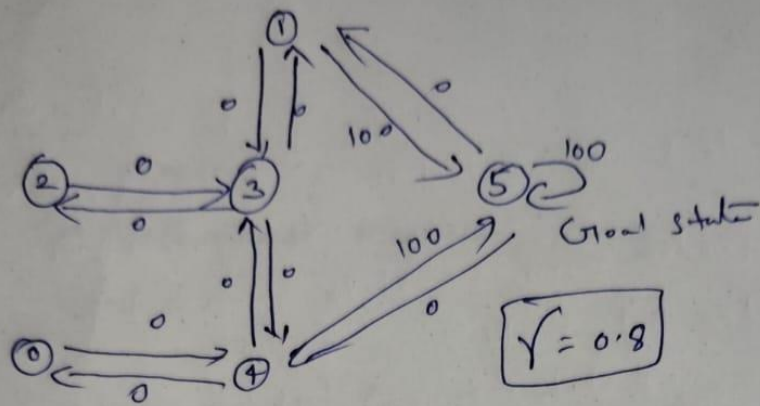
→ Do forever:
• Select an action a and execute it
• receive immediate reward r
When action is completed, move state s to s' .
• observe the new state s' .
• update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

r is immediate reward

γ is discount at Value

• $s \leftarrow s'$



→ we have states 0, 1, 2, 3, 4, 5
actions 0, 1, 2, 3, 4, 5

→ Assume state ③, $S=3$
we can perform action 1, 2, 4
perform action 1, goto 1
" " 2, goto 2
" " 4, goto 4
Select action ①, immediate reward is 0.

$$\hat{Q}(S, a) \leftarrow \pi + \gamma \max_{a'} \hat{Q}(S', a')$$

$$\hat{Q}(3, 1) \leftarrow 0 + 0.8 \max(0, 100) = 80$$

Now you are state 1, goto goal state.

$$\hat{Q}(1, 5) \leftarrow 100 + 0.8 \max(0, 0) = 100$$

Reached to goal state